

```
In [ ]: #K Nearest Neighbor Classifier Project for Week 2 Critical Thinking Option 1 Iris D
        #John Wensink
        #CSC525 Principles of Machine Learning
        #Colorado State University - Global Campus
        #Dr. Dong Nguyen
        #May 25, 2025
```

```
In [33]: #Cell 1 for data loading, inspection, feature/label separation
import pandas as pd

df = pd.read_csv("data/w2ct_iris.csv")

df["Name"].value_counts()
print(df["Name"].value_counts())

print("\nDataset Preview:")
print(df.head())

print("\nMissing values per column:")
print(df.isnull().sum())

features = df[["SepalLength", "SepalWidth", "PetalLength", "PetalWidth"]]
species = df["Name"]

print("\nFeature sample:")
print(features.head())

print("\nLabel sample:")
print(species.head())

#(GeeksforGeeks, 2025)
```

```
Name
Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
Name: count, dtype: int64
```

Dataset Preview:

	SepalLength	SepalWidth	PetalLength	PetalWidth	Name
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Missing values per column:

```
SepalLength  0
SepalWidth   0
PetalLength  0
PetalWidth   0
Name         0
dtype: int64
```

Feature sample:

	SepalLength	SepalWidth	PetalLength	PetalWidth
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Label sample:

```
0    Iris-setosa
1    Iris-setosa
2    Iris-setosa
3    Iris-setosa
4    Iris-setosa
Name: Name, dtype: object
```

```
In [34]: #Cell 2 will import sklearn's StandardScaler preprocessing module and put our data

#We want to normalize our z-score for this dataset so that our relatively large len
#We can standardize our data by rescaling absolute measurements as a function of av
#Since our dataset is purely numeric, with few outliers, and has a roughly Gaussian
from sklearn.preprocessing import StandardScaler as ss

#Declare the variable scaler as an instance of StandardScaler
scaler = ss()
#Declare the variable features_scaled as the standardized version of features using
features_scaled = scaler.fit_transform(features)

#Let's make a DataFrame to check our progress showing the first, middle, and last 3
scaled_df = pd.DataFrame(features_scaled, columns=features.columns)

#Find the middle rows of our dataset and declare it as middle_rows
middle_index = len(scaled_df) // 2
middle_rows = scaled_df.iloc[middle_index - 1 : middle_index + 2]
```

```
#Make a concatenated DataFrame showing the 3 first, middle, and last rows, and disp
pd.concat([
    pd.DataFrame(features_scaled, columns=features.columns).head(3),
    pd.DataFrame(middle_rows),
    pd.DataFrame(features_scaled, columns=features.columns).tail(3)
])
```

Out[34]:

	SepalLength	SepalWidth	PetalLength	PetalWidth
0	-0.900681	1.032057	-1.341272	-1.312977
1	-1.143017	-0.124958	-1.341272	-1.312977
2	-1.385353	0.337848	-1.398138	-1.312977
74	0.674501	-0.356361	0.307833	0.133226
75	0.916837	-0.124958	0.364699	0.264699
76	1.159173	-0.587764	0.592162	0.264699
147	0.795669	-0.124958	0.819624	1.053537
148	0.432165	0.800654	0.933356	1.447956
149	0.068662	-0.124958	0.762759	0.790591

In [62]:

```
#Cell 3 we will handle our train/test split using sklearn's train_test_split (Sciki
from sklearn.model_selection import train_test_split as tts

#A standard 80/20 split seems appropriate given that our data is already normalized
features_train, features_test, species_train, species_test = tts(
    features_scaled, species, test_size=0.2
    #, random_state=13 #uncomment for reproducibility or leave commented for random
)

# Confirm that the training and testing sets have the correct shapes
print("Training features shape:", features_train.shape)
print("Testing features shape:", features_test.shape)
print("Training target shape:", species_train.shape)
print("Testing target shape:", species_test.shape)

# Show class distribution in training and test sets
print("\nClass distribution in training set:")
print(species_train.value_counts())

print("\nClass distribution in test set:")
print(species_test.value_counts())
```

Training features shape: (120, 4)
 Testing features shape: (30, 4)
 Training target shape: (120,)
 Testing target shape: (30,)

Class distribution in training set:

Name	
Iris-virginica	41
Iris-versicolor	40
Iris-setosa	39

Name: count, dtype: int64

Class distribution in test set:

Name	
Iris-setosa	11
Iris-versicolor	10
Iris-virginica	9

Name: count, dtype: int64

```
In [64]: #Cell 4 we will introduce the k-NN classifier from sklearn, we'll import the classi
#Even though our features are numerical, our target is still categorical and as suc
from sklearn.neighbors import KNeighborsClassifier as knn #(Scikit-Learn, n.d. -d)

#Instantiate and fit the k-NN model
model = knn(n_neighbors=5) #ALL other hyperparameters use default values (metric=mi
model.fit(features_train, species_train)

#Display the model's hyperparameters
print("k-NN Configuration:")
print(model.get_params())
```

k-NN Configuration:

```
{'algorithm': 'auto', 'leaf_size': 30, 'metric': 'minkowski', 'metric_params': None,
'n_jobs': None, 'n_neighbors': 5, 'p': 2, 'weights': 'uniform'}
```

```
In [67]: #Cell 5 we will evaluate the trained k-NN model using the testing data. We'll use t
#Sklearn's accuracy_score is used to evaluate the model, it measures how often the
from sklearn.metrics import accuracy_score #(Scikit-Learn, n.d. -a)

#Just for fun Let's also run a classification_report to measure precision, recall,
from sklearn.metrics import classification_report #Scikit-Learn, n.d.)

#Make predictions on the test features
kNN_guess = model.predict(features_test)

#Evaluate the model's prediction accuracy
accuracy = accuracy_score(species_test, kNN_guess)

#Print the results for accuracy
print(f"Model accuracy on test data: {accuracy: .3f}")

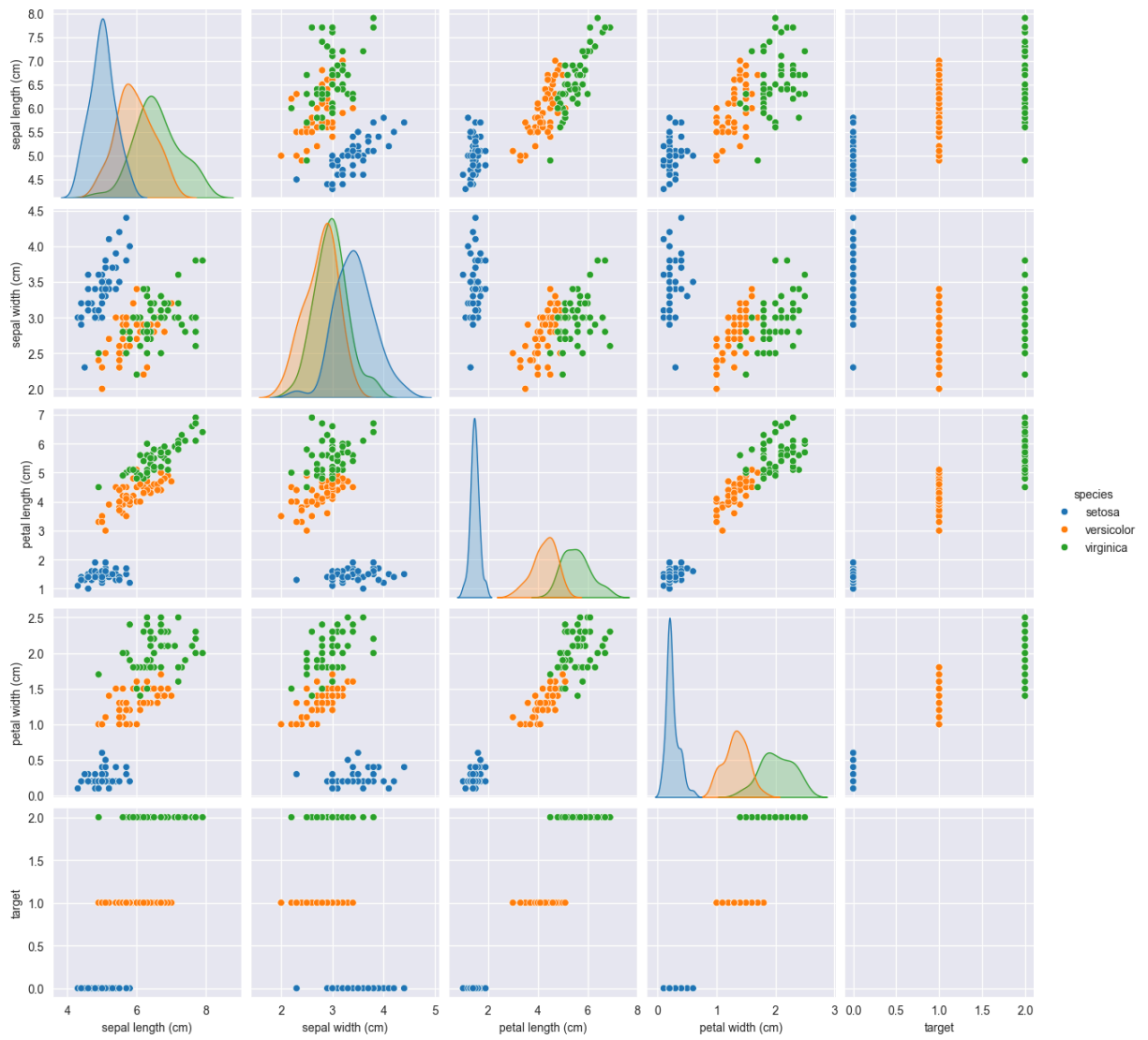
#Print the classification report results
print("\nClassification report:")
print(classification_report(species_test, kNN_guess))
```

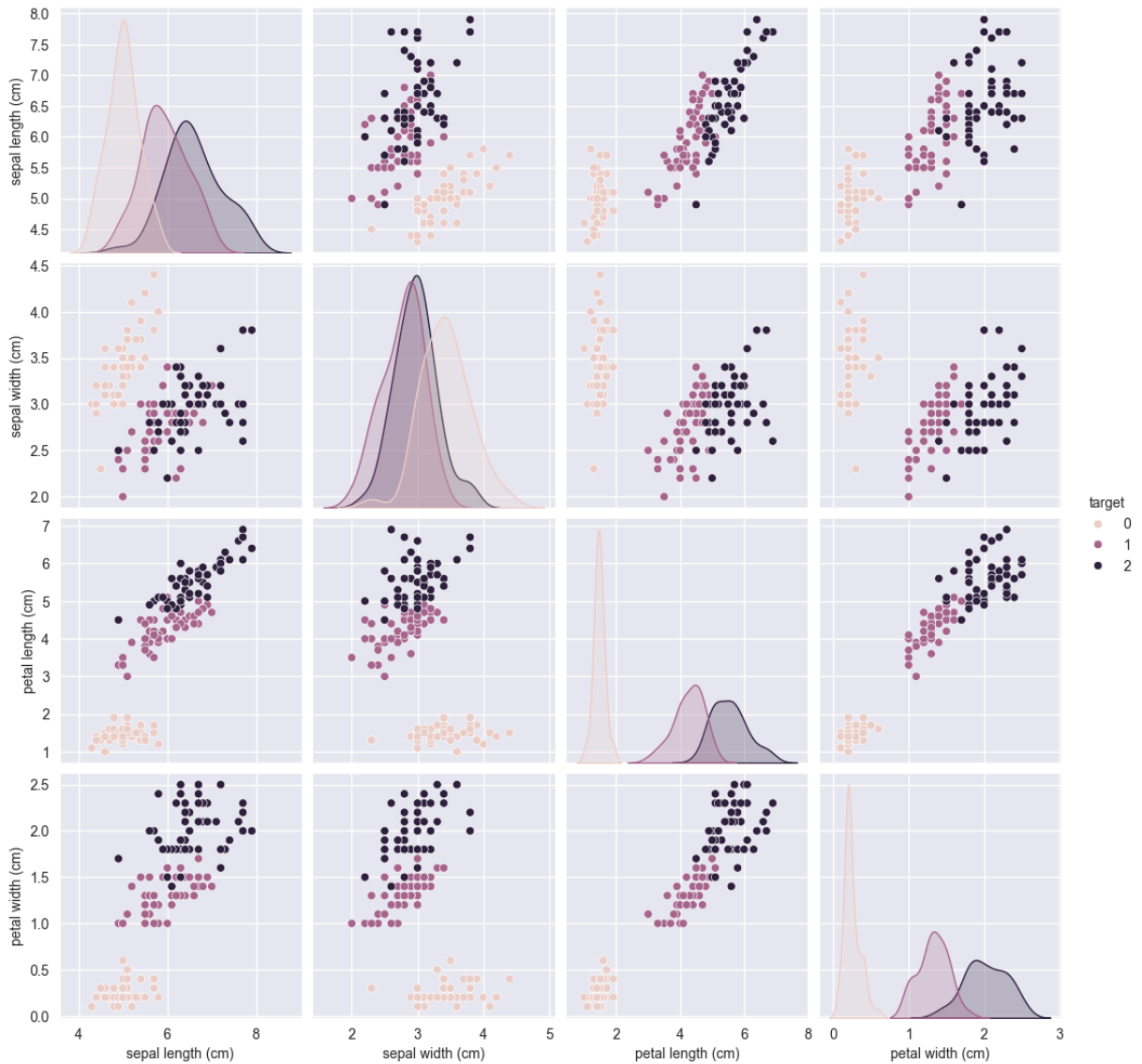
Model accuracy on test data: 0.967

Classification report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	11
Iris-versicolor	1.00	0.90	0.95	10
Iris-virginica	0.90	1.00	0.95	9
accuracy			0.97	30
macro avg	0.97	0.97	0.96	30
weighted avg	0.97	0.97	0.97	30

In [125... *#Cell 6 Import our textbooks visualization of the iris data from chapter 3 (Fenner,*
import **importlib**
import IrisData_Visualization **as** plots
 _ = **importlib**.**reload**(plots) *#Reloads the library _ = prevents displaying the path*
#These two pair plots from the iris dataset visualize how the features relate to on
#The top plot shows how sepal and petal dimensions vary by species name, using colo
#The bottom plot is essentially the same data, but colored by target integer instea





```
In [122... # Cell 7: Visualize k-NN classification performance using a confusion matrix and ba
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Generate the confusion matrix to compare predicted vs. actual labels
cm = confusion_matrix(species_test, kNN_guess, labels=model.classes_)

# Plot the confusion matrix as a heatmap (Scikit-Learn, n.d. -c)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=model.classes_, yticklabels=model.classes_)
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.show()

#Print raw confusion matrix with labeled columns.
print("\nRaw Confusion Matrix:")
```

```

header = " " * 18 + " ".join(f"{label:>15s}" for label in model.classes_)
print(header)
for actual_label, row in zip(model.classes_, cm):
    row_str = " ".join(f"{val:>15d}" for val in row)
    print(f"{actual_label:>15s} {row_str}")

#Extract precision, recall, and F1-score for each class from the classification rep
report = classification_report(species_test, kNN_guess, output_dict=True)
metrics = ['precision', 'recall', 'f1-score']
classes = model.classes_

#Organize scores for each metric and class into a dictionary
scores = {metric: [report[label][metric] for label in classes] for metric in metrics}

#Prepare to plot grouped bar charts
x = range(len(classes))
width = 0.25

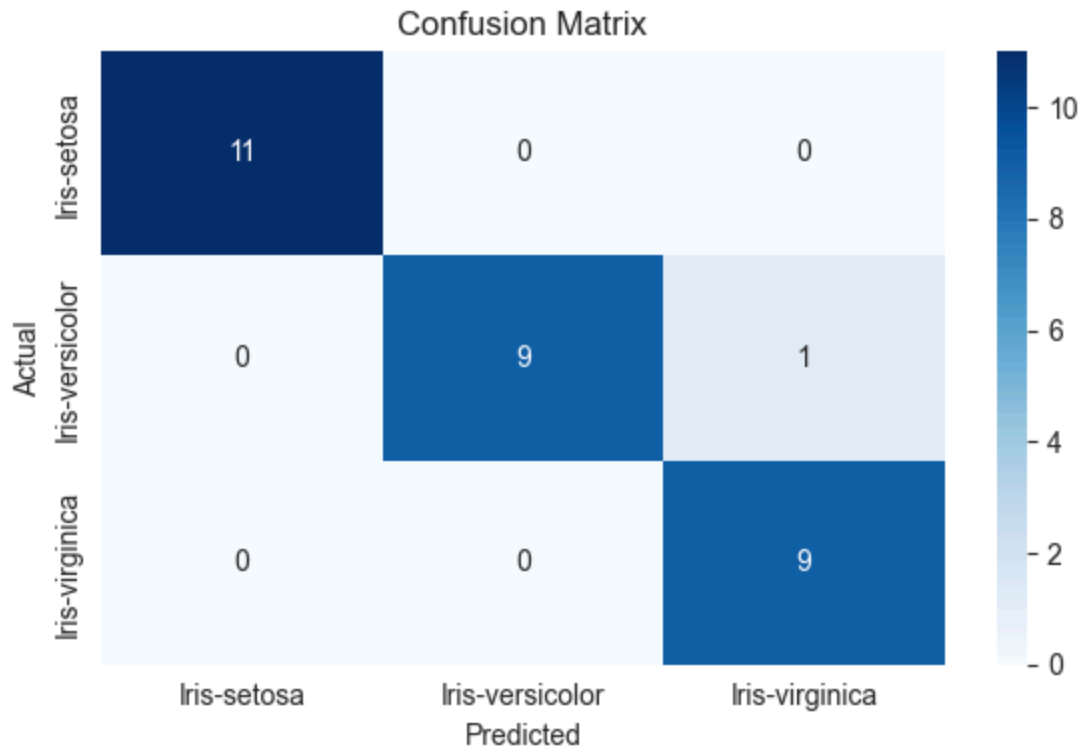
#Bar chart construction using Matplotlib
plt.figure(figsize=(8, 5))
for i, metric in enumerate(metrics):
    plt.bar([p + width * i for p in x], scores[metric], width=width, label=metric)

# Configure axis labels and layout
plt.xticks([p + width for p in x], classes)
plt.ylim(0, 1.1)
plt.ylabel("Score")
plt.title("Precision, Recall, and F1-Score by Class")
plt.legend()
plt.tight_layout()
plt.show()

#Output the text scores used to make the plot
print("Scores used for plotting:\n")
for metric in metrics:
    print(f"{metric.capitalize()}:")
    for cls, score in zip(classes, scores[metric]):
        print(f" {cls}: {score:.2f}")
    print() # Blank line for readability

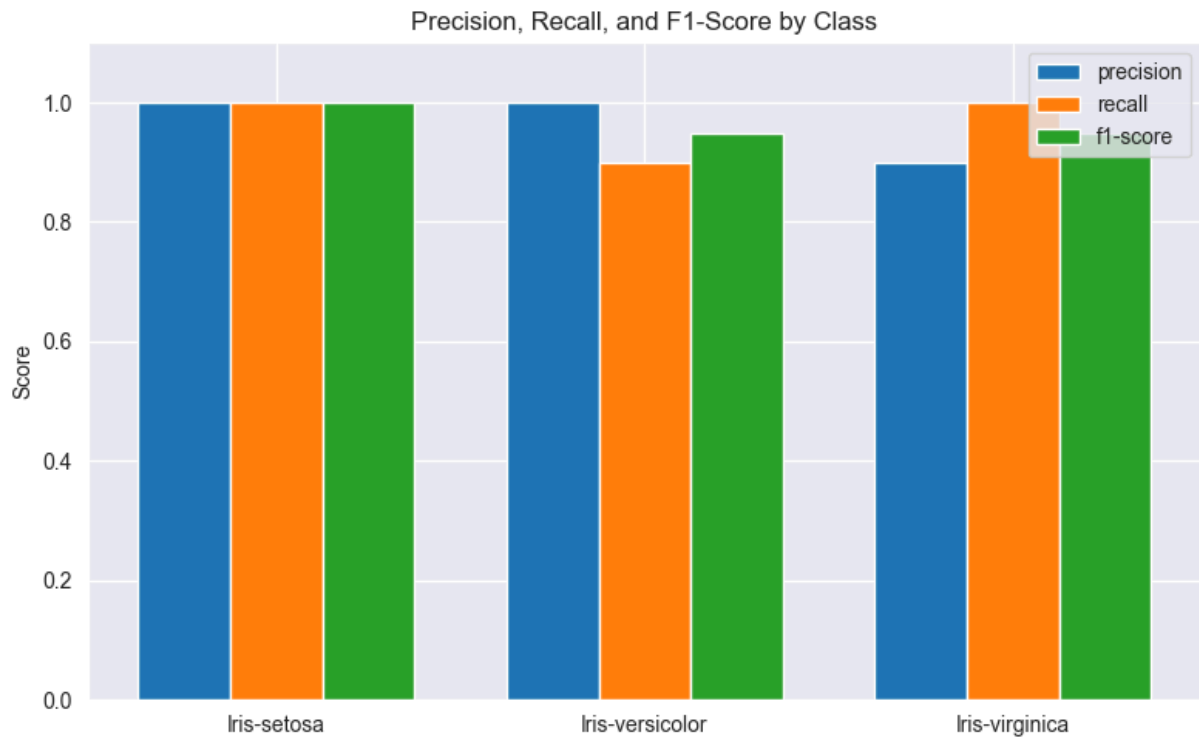
#Given the separation of the classes in these pair plots, we should expect the clas

```

Raw Confusion Matrix:

	Iris-setosa	Iris-versicolor	Iris-virginica
Iris-setosa	11	0	0
Iris-versicolor	0	9	1
Iris-virginica	0	0	9



Scores used for plotting:

Precision:

Iris-setosa: 1.00
Iris-versicolor: 1.00
Iris-virginica: 0.90

Recall:

Iris-setosa: 1.00
Iris-versicolor: 0.90
Iris-virginica: 1.00

F1-score:

Iris-setosa: 1.00
Iris-versicolor: 0.95
Iris-virginica: 0.95

```
In [36]: #References:
#Fenner, M. E. (2020). Machine Learning with Python for everyone. Pearson Education

#GeeksforGeeks. (2025, May 2). Exploratory Data Analysis on Iris Dataset.
#https://www.geeksforgeeks.org/exploratory-data-analysis-on-iris-dataset/

#Grimoire. (2025). OpenAI Grimoire GPT assistant [AI LLM].
#https://apps.apple.com/us/app/hivemind-grimoire/id6446332488

#Scikit-Learn. (n.d. -a). Accuracy_score. Scikit-Learn documentation.
#https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.h

#Scikit-Learn. (n.d. -b). ClassificationReport. Scikit-Learn documentation.
#https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_r

#Scikit-Learn. (n.d. -c). ConfusionMatrixDisplay. Scikit-Learn documentation.
#https://scikit-learn.org/stable/modules/generated/sklearn.metrics.ConfusionMatrixD

#Scikit-Learn. (n.d. -d). KNeighborsClassifier. Scikit-Learn documentation.
#https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClas

#Scikit-Learn. (n.d. -e). StandardScaler. Scikit-Learn documentation.
#https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardSc

#Scikit-Learn. (n.d. -f). TrainTestSplit. Scikit-Learn documentation.
#https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_te
```